

Differentiable Image Data Augmentation And Its Applications: A Survey

Jian Shi, Hakim Ghazzai, *Senior Member, IEEE* and Yehia Massoud, *Fellow, IEEE*

Abstract—Data augmentation is an effective method to improve model robustness and generalization. Conventional data augmentation pipelines are commonly used as preprocessing modules for neural networks with predefined heuristics and restricted differentiability. Some recent works indicated that the differentiable data augmentation (DDA) could effectively contribute to the training of neural networks and the augmentation policy searching strategies. Some recent works indicated that the differentiable data augmentation (DDA) could effectively contribute to the training of neural networks and the searching of augmentation policy strategies. This survey provides a comprehensive and structured overview of the advances in DDA. Specifically, we focus on fundamental elements including differentiable operations, operation relaxations, and gradient estimations, then categorize existing DDA works accordingly, and investigate the utilization of DDA in selected of practical applications, specifically *neural augmentation networks* and *differentiable augmentation search*. Finally, we discuss current challenges of DDA and future research directions.

Index Terms—Computer Vision, Data Augmentation, Differentiability

1 INTRODUCTION

MODERN deep neural networks heavily rely on data augmentations techniques, which are widely applied to increase the amount of data by transforming the original data or synthesizing from the existing data. Conventionally, data augmentation can effectively alleviate overfitting to improve the model generalization in a low-data regime. For optimal model performances, curated data augmentation strategies are preferred as they may affect downstream tasks [1]. In computer vision, transformation-based augmentation includes color space transformations that modify pixel intensity values (e.g. brightness, contrast adjustment) and geometric transformations that update the spatial locations of pixels (e.g. affine transformations), while synthetic-based augmentation resorts to generative methods (e.g. neural style transfer [2]), adversarial methods (e.g. adversarial training [3]), and mixing augmentations (e.g. MixUp [4], CutMix [5]). Those data augmentation methods are developed to effectively improve the model performances for vision tasks.

Furthermore, recent advances may perform data augmentation by synthesising new images through the learnt decoupled feature representations (e.g. InfoGAN [6]) and neural rendering [7]. Additionally, latent-space augmentations (e.g. Manifold MixUp [8], MODALS [9]) performed data augmentation in feature spaces, while negative data augmentation [10] incorporated the prior knowledge of bad examples by creating out-of-distribution data. Meanwhile, studies on self-supervised and semi-supervised learning focused on leveraging the content similarities between the weak-augmented and strong-augmented samples to learn the invariant feature representations of given images. Studies such as MixMatch [11], ReMixMatch [12], and

FixMatch [13] are semi-supervised frameworks that focus on augmentation anchoring, a method that leverages the augmentation consistency, to improve the performance of the target model. Additionally, many contrastive learning paradigms (e.g. SimCLR [14], MoCo [15]) are designed to learn the general feature representations by closing a positive pair (e.g. an image with different augmentations) and distancing a negative pair (e.g. different images), while BYOL [16] and SimSiam [17] further simplify the paradigm to learn without negative pairs. Despite their effectiveness, most of these common data augmentation techniques rely on non-differentiable functions executed outside the computation graphs.

Gradient-based optimization methods have brought dramatic advances in applications such as deep learning, while image transformation gradients are the most common neglected ingredient during the training of neural networks. The incorporation of DDA and neural networks has been introduced to maintain the augmentation gradient flow. Thus, the computed augmentation gradients can be further integrated into the neural network optimization graph. Essentially, differentiable data augmentation (DDA) refers to the image transformations that are differentiable with respect to (w.r.t.) the input image, while it would further be optimizable if the transformations are differentiable w.r.t the transformation parameters. For example, affine transformations like scaling, rotation, translation etc. are differentiable operations. So that we can compute gradients of the output image w.r.t. the input image and the transformation parameters. To enable the gradient of image transformation operators, Kornia [18] implemented most differentiable image operators in PyTorch, while Halide [19] proposed a domain-specific language for differentiable image processing which claims to be a high-performance image processing architecture compared to PyTorch and manual CUDA programming. In this work, a DDA module that contains one or more DDA operations is referred as a *neural augmentation*

• The authors are with Innovative Technologies Laboratories (ITL), King Abdullah University of Science and Technology (KAUST), Thuwal, Saudi Arabia. Email: {jian.shi, hakim.ghazzai, yehia.massoud}@kaust.edu.sa

Manuscript received xxxxx xx, xxxx; revised xxxxx xx, xxxx.

network. There are two types: the *static neural augmentation network* that does not update augmentation parameters, and the *optimizable neural augmentation network* that does update augmentation parameters. Previous works tended to use *static neural augmentation network* to replace traditional image operations [20]–[22] as a network layer for GANs, or to replace routine non-differentiable data augmentation [23]–[25] for data-efficient learning. Meanwhile, several studies investigated augmentation policy searching [26]–[29] with *optimizable neural augmentation networks*. Apart from improving model training, various applications have been explored, such as mimicking camera sensors [30], generating underwater-like images from non-underwater images [31]. Also, an interesting research [32] integrated DDA to trace the gradient to detect if a particular image dataset has been used in training.

This survey is an attempt to provide a structured and broad overview of the recent works on DDA, spanning many research and application domains. Previous surveys, Shorten *et al.* [33] and Xu *et al.* [34], have comprehensively investigated augmentation techniques, including traditional augmentation transformations, GAN-based synthetic methods, and automatic augmentation algorithms. Yang *et al.* [35] further elaborated automatic augmentation methods in details. Naveed [36] focused on image mixing and deleting methods. To the best of our knowledge, this is the first survey that is specifically focusing on DDA. In summary, our contributions are as follows:

- **Comprehensive literature review.** We introduce DDA and provide a comprehensive literature review covering the recent and relevant studies to assess the current state of the art of DDA.
- **Categorization.** We categorize the different DDA techniques and present their different operations and ways of implementation.
- **Application.** We classify the current DDA applications into two principled paradigms: differentiable neural augmentation and differentiable augmentation search.
- **Challenges and future opportunities.** We discuss the challenges and future opportunities of DDA.

The paper is organized as follows. Section II introduces the concept of DDA, along with the differentiable image transformation operators as well as the common gradient approximation methods. Afterwards, in Section III and IV, we elaborate the applications of *differentiable neural augmentation* and *differentiable augmentation search*, respectively. In Section IV, we discuss the challenges and opportunities of DDA. Finally, Section V concludes our paper.

2 DIFFERENTIABLE DATA AUGMENTATION

This section presents the concept of DDA and summarizes common image augmentation operations.

2.1 Prerequisites

Image data augmentation is a composition of image processing operators to increase the diversity of a given dataset. Apart from the neural-network-based augmentations, most common data augmentation techniques are non-differentiable and executed outside the computation graphs.

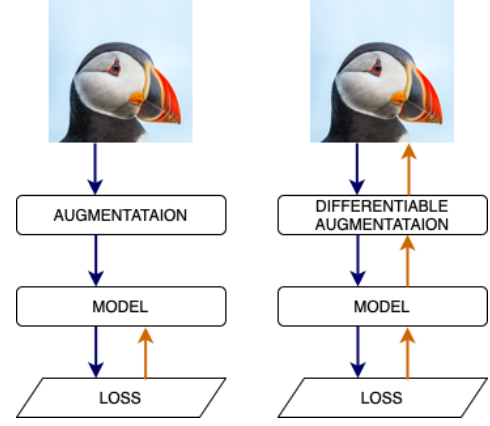


Fig. 1: A high-level comparison between traditional and differentiable augmentation methods. Purple and orange arrows denote forward and backward operations, respectively.

Thus, to formally introduce differentiable data augmentation, we borrow the definition of *stochastic computation graph* and the requirements of *differentiability* from Schulman *et al.* [37]:

Definition 1. Stochastic Computation Graph. A directed, acyclic graph, with three types of nodes:

- 1) Input nodes, which are set externally, including the parameters we differentiate w.r.t.
- 2) Deterministic nodes, which are functions of their parents.
- 3) Stochastic nodes, which are distributed conditionally on their parents.

Each parent v of a non-input node ω is connected to it by a directed edge (v, ω) .

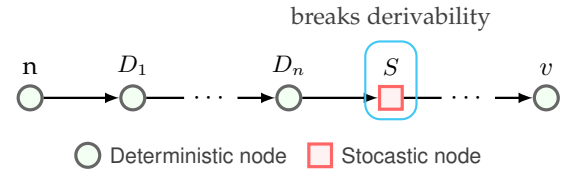


Fig. 2: Demonstration of non-differentiable path from n to v .

Within a stochastic computation graph $\mathcal{N}$, if the path from an input node n to a deterministic node v passes through stochastic nodes, then v may be a non-differentiable function of its parent nodes, since the stochastic nodes can introduce discontinuities that break derivability, as illustrated in Fig. 2. Formally, given an input node $n \in \mathcal{N}$, for all edges (v, ω) which satisfy $n \prec^D v$ and $n \prec^D \omega$, then the following condition holds: if ω is deterministic, Jacobian $\frac{\partial \omega}{\partial v}$ exists, and if ω is stochastic, then the derivative of the probability mass function $\frac{\partial}{\partial v} p(\omega | \text{PARENTS}_\omega)$ exists. Note that the notation of $n \prec^D v$ (n deterministically influences v where D represents deterministic nodes) means a deterministic path flow from n to v exists. In summary, DDA refers to a set of differentiable image processing algorithms, that can be interpreted as a stochastic computation graph to compute gradients w.r.t. the image. Essentially, as illustrated in Fig. 1, DDA enables the gradient flow through augmentation operations.

2.2 Differentiable Image Transformation

As fundamental ingredients of DDA, differentiable image transformations¹ have been widely applied in many 2D and 3D computer vision tasks, including image restoration [23], colorization [24], [25], and differentiable rasterization [38]–[40]. This section categorizes common image transformation operators, with a focus on elaborating their differentiability. Specifically, we cover 1) geometric transformations, 2) intensity transformations, and 3) mix transformations. Table 1 summarizes common operations for image data augmentation, noting their differentiability. Additionally, we include *neural transformations* that are neural network-based transformation operators.

TABLE 1: A summary of common image transformation operations and their differentiability. Linear operations are differentiable operations, while brief explanations are provided in the *comments* column for non-linear operations.

category	Operation	Diff.	Comments
Geometric	crop	✓	w/ perspective transform
	flip	✓	
	identity	✓	
	perspective	✓	
	rotate	✓	
	scale	✓	
	shear	✓	
	translate	✓	
Intensity	brightness	✓	
	contrast	✓	
	equalize	×	discrete LUTs
	invert	✓	
	posterize	×	uint8 operations
	sharpness	✓	
	solarize	✓	not diff. w.r.t its parameter
Mix	cutmix	×	coordinate selection
	cutout	×	coordinate selection
	mixup	✓	
	sample_pairing	✓	

2.2.1 Geometric Transformations

In general, geometric transformations map each pixel coordinate (x, y) in an image to a new location $(\hat{x}, \hat{y})$. In order to perform differentiable geometric transformations, Jaderberg et al. [41] introduced a differentiable image sampling method. To transform a given image in a shape of $H \times W \times C$, to the target shape $\hat{H} \times \hat{W} \times C$:

$$\hat{V}_i^c = \sum_n^H \sum_m^W V_{nm}^c k(x_i - m | \Phi_x) k(y_i - n | \Phi_y), \quad (2.1)$$

$$\forall i \in [1 \dots \hat{H}\hat{W}] \quad \forall c \in [1 \dots C],$$

where $k(\cdot | \Phi)$ denotes the image interpolation (e.g. bilinear) parameterized by Φ , while V_{nm}^c and $\hat{V}_i^c$ correspond to the input values at (n, m, c) location and the i^{th} output value in channel c , respectively. Note that not all interpolation kernels are differentiable. For example, bilinear interpolation is

differentiable but not nearest-neighbor interpolation. With bilinear interpolation, the partial derivatives are:

$$\frac{\partial \hat{V}_i^c}{\partial V_{nm}^c} = \sum_n^H \sum_m^W \max(0, 1 - |x_i - m|) \max(0, 1 - |y_i - n|), \quad (2.2)$$

$$\frac{\partial \hat{V}_i^c}{\partial x_i} = \sum_n^H \sum_m^W \partial \hat{V}_{nm}^c \max(0, 1 - |y_i - n|) \begin{cases} 0 & \text{if } |m - x_i| \geq 1, \\ 1 & \text{if } m \geq x_i, \\ -1 & \text{if } m < x_i \end{cases} \quad (2.3)$$

and same expressions as in (2.3) can be derived for $\frac{\partial \hat{V}_i^c}{\partial y_i}$. The whole pipeline backpropagates the gradients from the transformed images to the input images via the sampled grid coordinates.

With the rationale of getting the target transforming coordinates, we cover common geometric transformations that are used as data augmentation techniques, such as affine transformation, perspective transformation, and elastic transformation. Those transformations can be implemented in a differentiable manner taking the advantage of (2.1). Fig. 3 summarizes common affine and perspective transformation matrices. For further details regarding the differentiable implementation of these transformations, refer to [18].

Affine Transformation is a geometric transformation that preserves points, straight lines, and planes by linear mappings with affine transformation matrices. Formally, the affine transformation matrix M for image transformations can be defined as:

$$\begin{bmatrix} \hat{x}_i \\ \hat{y}_i \\ 1 \end{bmatrix} = H \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} & b_{11} \\ A_{21} & A_{22} & b_{21} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}, \quad (2.4)$$

where (x_i, y_i) and $(\hat{x}_i, \hat{y}_i)$ represent the coordinates in an image before and after transformation, respectively, and H represents a homography. Affine operations apply restricted 6-DOF homographies to preserve parallelism of lines. To be specific, a homography is decomposed into matrices A and b , in which A is a 2×2 matrix to relocate pixels and b is a two-elements translation vector. Affine transformations can be inverted by applying the inverted matrix H^{-1} . To chain on n affine transformations, one may simply apply $H = H_n \dots H_1$.

Perspective Transformation uses homogeneous coordinates in the projective space, that does not preserve the line parallelism as affine transformations. Formally, a homography is a more generic transform representation with an 8-DOF, than the constrained 6-DOF affine transformations. It can be expressed as:

$$\begin{bmatrix} \hat{x}_i \\ \hat{y}_i \\ 1 \end{bmatrix} = H \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}. \quad (2.5)$$

Notably, cropping is a transformation that extracts a sub-area of a given image by removing the pixels at the sides. It is a non-differentiable index-based operation that consumes the start and end coordinates to select desired regions. Thus, perspective transformations come as a rescue to obtain meaningful gradients for the non-differentiable cropping [42].

Elastic Deformation is introduced to create the deformations that mimics uncontrolled oscillations [43], which have

¹ Images would normally be required to be mapped into a floating point space ranged from 0 to 1 instead of using 8-bit unsigned integer *uint8* that ranged from 0 to 255 due to its discontinuity nature.

been reported as a useful augmentation in areas like medical imaging [44], [45]. Other than performing transformations with transformation matrices, Simard *et al.* [43] proposed to generate randomized locations for each pixel coordinate w.r.t. the original location with:

$$\begin{aligned}\Delta x &= \text{Gaussian}(\text{rand}(-1, 1)|k, \sigma), \\ \Delta y &= \text{Gaussian}(\text{rand}(-1, 1)|k, \sigma),\end{aligned}\quad (2.6)$$

where $\text{rand}(\cdot)$ is a uniform sampler and $\text{Gaussian}(\cdot|k, \sigma)$ is a Gaussian filter to smooth the random generated new coordinates parameterized by the kernel size k and elasticity coefficient σ .

Identity	Horizontal Flip	Translate
$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & w-1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix}$
Shear	Vertical Flip	Scale
$\begin{bmatrix} 1 & \lambda_x & 0 \\ \lambda_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & h-1 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$
Rotation	Perspective	
$\begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}$	

Fig. 3: A summary of the common used transformation matrices.

2.2.2 Intensity Transformation

Intensity transformation refers to the image processing operations that modify the pixel intensities without changing its geometric locations. In general, intensity transformation includes *point operations* that transform each pixel independently, and *local operations* that transform each pixel referring to its neighbors. This survey follows [46] regarding the definition of each operation, since there are diverged versions of color enhancement transformations.

Point Operation transforms each pixel w.r.t its previous state. With $f(x_i, y_i)$ and $g(x_i, y_i)$ represent the input and output pixel values in location (x_i, y_i) and T is the transformation function. The generic point operation can be expressed as:

$$g(x_i, y_i) = T(f(x_i, y_i)). \quad (2.7)$$

In the following, we list the transformations that use the naive 1×1 neighborhood size:

$$\begin{aligned}\text{contrast} \quad & g(x_i, y_i) = a \cdot f(x_i, y_i) \quad a > 0, \\ \text{brightness} \quad & g(x_i, y_i) = f(x_i, y_i) + b, \\ \text{gamma} \quad & g(x_i, y_i) = k \cdot f(x_i, y_i)^g, \\ \text{solarize} \quad & g(x_i, y_i) = \begin{cases} f(x_i, y_i) & \text{if } f(x_i, y_i) < k, \\ 1 - f(x_i, y_i) & \text{if } f(x_i, y_i) \geq k \end{cases}, \\ \text{posterize} \quad & g(x_i, y_i) = \lfloor \frac{f(x_i, y_i)}{2^p} \rfloor \cdot 2^p, \quad p \in [0 \dots 8],\end{aligned}$$

where a, b, g, k , and p are the magnitudes for each corresponding transformation, and $\lfloor \cdot \rfloor$ rounds the images with 8-bit unsigned integer *uint8* (0 to 255). *Posterize* is non-differentiable since it only accepts *uint8* inputs for non-linear *bit shift* operations.

Another operation, identified as *Equalize*, processes images in order to adjust its contrast by modifying the intensity distribution of the image histogram, which transforms each pixel to a high contrast according to the calculated image histogram. The general histogram equalization formula is:

$$g(x, y) = \lfloor \left(\frac{\text{cdf}(f(x, y)) - \text{cdf}_{\min}}{(h \times w) - \text{cdf}_{\min}} \right) \times L \rfloor, \quad (2.8)$$

where *cdf* refers to the cumulative distribution, $\text{cdf}_{\min}$ is the minimum non-zero value of the cumulative distribution function, L is the maximum intensity value (255 if *uint8*), h and w are the image height and width, respectively. Technically, *equalize* is implemented with non-differentiable Look-Up-Tables (LUTs). To the best of our knowledge, no valid differentiable *equalize* methods have been implemented, though some differentiable histogram calculation methods have been proposed [47]–[49].

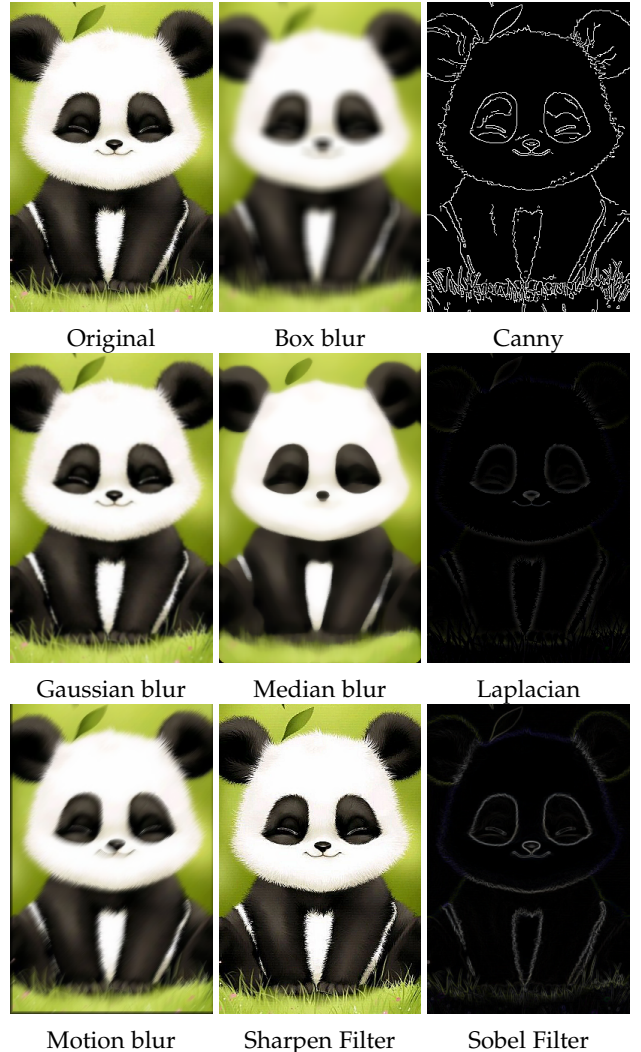


Fig. 4: Demonstration of the effect of common filtering operations. Kernel size is set to 13 for all filters.

Local Operation refers to the operations that transform each pixel that considers a collection of pixel values. Its differentiability depends on the differentiability of the targeted local operators. For example, an intuitive local operation that uses *linear filters* to update each pixel by a weighted sum of pixel values within a neighborhood can be expressed as follows:

$$g(x_i, x_j) = \sum_{k,l} f(x_{i+k}, y_{j+l})h(x_k, x_l), \quad (2.9)$$

where $h(x_k, x_l)$ represents the weight for each neighbor pixel around in $f(x_i, x_j)$. Similarly, *linear filtering operations* perform convolution operations which are apparently differentiable. Contrarily, non-continuous local operations are non-differentiable since they involve discontinuities or singularities. For instance, *median filter* is a typical non-differentiable operation that operates in a discrete manner. Fig. 4 illustrates common filtering operations.

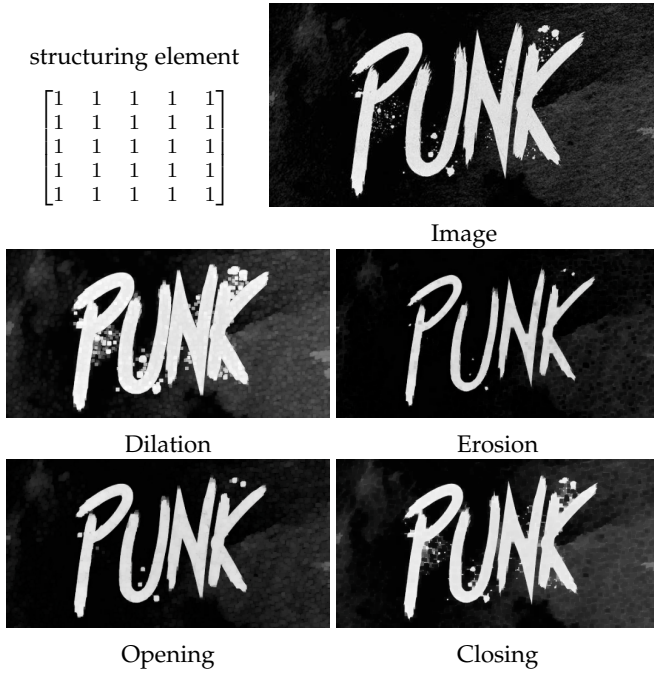


Fig. 5: Demonstration of the effect of morphological transformations.

Morphological transformation contains two fundamental operators: *erosion* and *dilation*. *Erosion* operation can diminish the size the foreground object, while the *dilation* operation expands the shape in the images. Particularly, a structuring element is a key concept in morphological transformations that is a shape (e.g. squares, circles, diamonds) used to probe an image to selectively expand or shrink features based on brightness, shape, and size constraints. Given an input image X and a structuring element B , the *dilation* and *erosion* operators can be written as follows:

$$\begin{aligned} \text{Dilation} \quad X \oplus B &= \{z | [\hat{B}_z \cap X] \in X\}, \\ \text{Erosion} \quad X \ominus B &= \{z | \hat{B}_z \in X\}, \end{aligned} \quad (2.10)$$

where $\hat{B}_z$ is the translation of B by the vector z , allowing the structuring element B to be translated or shifted to different positions across the image.

Technically, the *erosion* operation computes the pointwise minimum of the given image and structuring element, while the *dilation* takes the pointwise maximum. In terms of differentiability, the intuitive approach is to perform the minimum/maximum comparison directly with less meaningful gradients, whereas one may work around with convolution operations for more meaningful gradients but less stable numerically. For the detailed implementations, refer to Kornia [18], as well as a CUDA version². Furthermore, morphological *opening* operator $\circ$ and *closing* operator $\bullet$ can be differentiable by stacking the *erosion* and *dilation* processes and can be expressed as follows:

$$\begin{aligned} \text{Opening} \quad X \circ B &= (X \ominus B) \oplus B, \\ \text{Closing} \quad X \bullet B &= (X \oplus B) \ominus B. \end{aligned} \quad (2.11)$$

Fig. 5 presents a visual demonstration for common morphological operations.

2.2.3 Mix Transformation

Mix transformations transform each image with a collection of images, that improves model generalization by generating "out-of-distribution" examples to prevent the overfitting on the training distribution. Mix augmentations have been extensively reviewed by [50]. We would hereby cover some representative methods, showcased in Fig. 6, and discuss their differentiability.

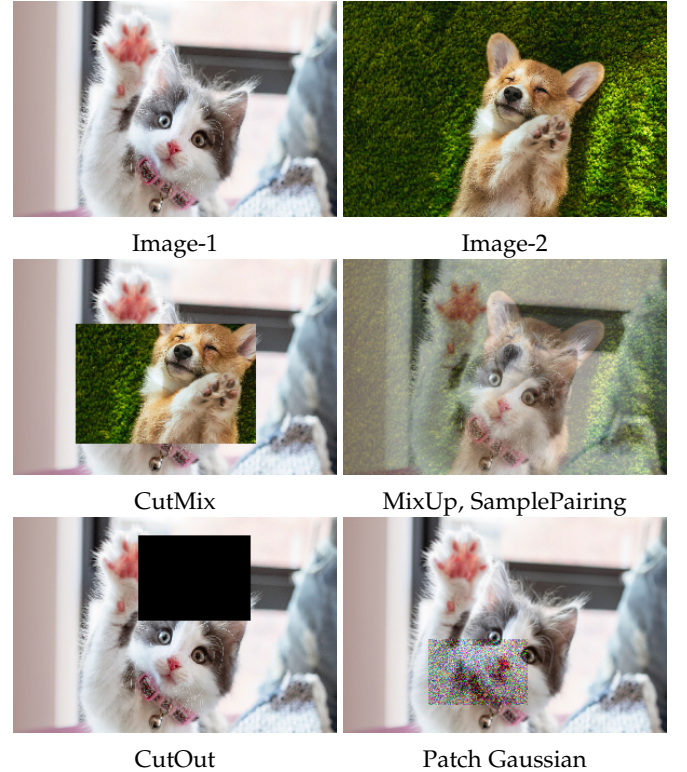


Fig. 6: Demonstration of the effect of common Mixing operations. Note that SamplePairing does not change the label information whilst computing losses, but MixUp does.

SamplePairing [51] mixes two cropped patches from two randomly selected images (A and B), then averages the two

². <https://github.com/Manza12/nnMorpho.git>

patches without modifying the label. This technique aims at improving the model robustness by confusing the network since the ground truth $0.5 \times A + 0.5 \times B$ can be either A or B , but the confusion also complicates the training procedures as stated by the authors. MixUp [4] applies a similar idea by interpolating the pixel values between two images by a weighted sum, and it further manipulates the label accordingly for Empirical Risk Minimization (ERM). These linear methods are obviously differentiable as they apply a simple $I_{out} = \alpha \times A + \beta \times B$ where α and β are the coefficients for each image. CutMix [5] randomly cuts out portions of an image and places them over another. Mosaic [52] randomly cuts out and mixes four training images at one time. It also proposes to modify the label representations similarly to MixUp [4]. Additionally, it is worth to mention that we also include the image-noise mixing into this category that mixes up images and noises. CutOut [53] erases a square region of the input image by masking the corresponding pixel values with zeros. RandomErase [54] and PatchGaussian [55] are proposed to replace the masked area with random values (e.g. noise, ImageNet mean pixel values) and a Gaussian blurred region. The aforementioned mask-based and patch-based methods are non-differentiable since the coordinates are discrete values.

2.2.4 Neural Transformation

Neural network-based transformation operators are naturally differentiable since neural networks are inherently differentiable functions. Unlike traditional image operators, neural transformation operators do not require manually designed features or rules. Instead, they automatically learn the appropriate representations and transformations directly from data through end-to-end training. So, potential domain gaps may exist whilst applying neural transformation on a new dataset. Data generation techniques [56]–[60] are the most common neural transformation methods, that can be used as static neural augmentation networks (as in Section 3.1.1) to enlarge the dataset. In this survey, we particularly focus on the works that take advantage of the differentiability of neural networks that can be used as data augmentation operators to generate meaningful variants of a given image.

Image Editing. Generative models like GANs learn a latent space, that offers the possibility to control the generation outcome [6], [61]. Also, simple transformations like adding small noise to the latent codes can result in realistic augmented data. The generator is differentiable, so these small perturbations can be backpropagated through. For instance, DAGAN [62] and MineGAN [63] proposed to manipulate the latent space of generative models to generate augmented samples that significantly improved model performances under low-data regime. In particular, this area of research involves the inversion problem [64]–[66], that to investigate how to invert a given image back into the latent space of a pretrained generative model so that the image can be faithfully reconstructed from the inverted code by the generator. Many existing works [67], [68] utilized inversion techniques for data augmentation in various domains. Additionally, BlobGAN [69] proposed an interesting unsupervised representation learning method that decomposes a scene into blobs (*i.e.* the BlobGAN introduced latent space),

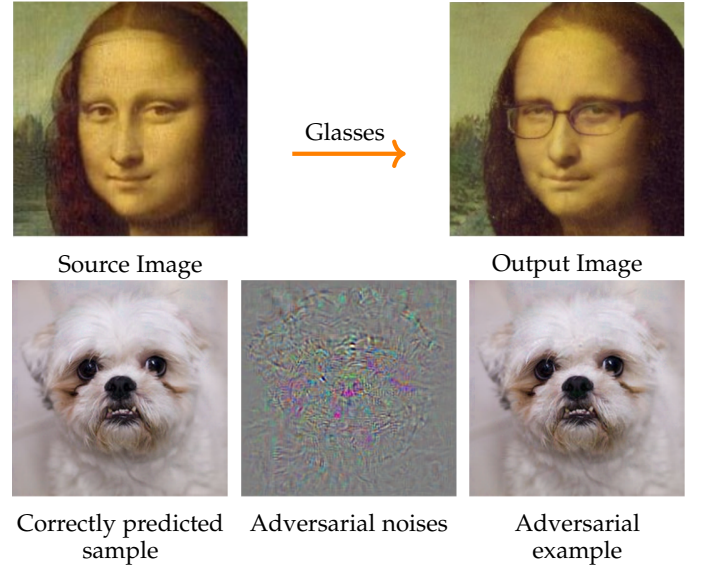


Fig. 7: Demonstration of neural network transformations. Upper: Semantic manipulation via *in-domain* GAN inversion technique [73]. Lower: Adversarial example taken from [74].

in which all the blobs are differentially placed onto a feature grid that is decoded into an image by a generative adversarial network. Though not widely explored yet, recent advances of guided image editing techniques [60], [70]–[72] may potentially be seen as data augmentation operators.

Adversarial Examples. The process of generating adversarial examples is fully differentiable with respect to the model parameters [74], [75], where the generated adversarial examples confuse the target model with little image perturbations. Adversarial training provides an elegant way to directly increase model robustness by learning on targeted adversarial examples generated through differentiation [76]–[78]. Classic works such as projected gradient descent (PGD) adversarial training is proposed by Madry *et al.* [3], that uses PGD as a reliable first-order adversary to improve model robustness. Additionally, virtual adversarial training (VAT) [79] adds small perturbations to the input that maximize the output distribution change, to improve model performance and robustness. The perturbation generation process is differentiable and done along with model training. To mention that adversarial attack and robustness problem is a hot area of research, interested readers may refer to [80], [81].

2.3 Gradient Approximation

As indicated in Table 1, non-differentiable transformations (e.g. posterize) exist in the common data augmentation routines. Most of the non-differentiable transformations create discontinuities with discrete sampling operations, resulting in zero gradient w.r.t. its parameters. An intuitive consideration is to redesign the computation graph into a differentiable manner as the cropping transformation example introduced in Section 2.2.1. However, not all transformations can be redesigned to be differentiable. Thus, in order to maintain the gradient flow, recent studies tend to construct gradient estimators to create optimizable surrogate function to re-

place the operation gradients with the estimated ones. Formally, in reinforcement learning, gradient estimation aimed at choosing the parameters θ of a distribution $\pi(a|s, \theta)$ to maximize the expected reward over state-action trajectories τ . By using the reparametrization trick [37] or other gradient estimators such as straight-through [82] or relax [83], the meaningful augmentation gradients can be computed and integrated into the neural network optimization graph. Next, we summarize common gradient estimators applied in DDA-related literature, including backpropagation-based methods and sampling-based methods.

2.3.1 Backpropagation-Based Methods

The backpropagation-based methods directly backpropagate gradients through non-differentiable operations, essentially “pretending” they are differentiable. Reparameterization computes analytical gradients that is more mathematically principled with a lower variance, while STE is a simpler heuristic that can be unstable.

Reparameterization often refers as the reparameterization trick that reconstructs the parameterized random variable to a parameterized deterministic function, that is independent of the initial parameter. As an example, we cite the *Gumbel Softmax* [84], which is a categorical reparameterization method, that uses softmax as a differentiable approximation to the *argmax* operation of Gumbel-Max trick [85]. Let z be a categorical variable (possibilities while differentiable), the Gumbel-Softmax is then given by:

$$\sigma_\eta(z) = \frac{\exp((\log(z_k) + g_k)/\eta)}{\sum \exp((\log(z_i) + g_i)/\eta)}, \quad (2.12)$$

where $g_1, \dots, g_k$ are random samples from Gumbel distribution and η is a temperature parameter. When η is small, $\sigma_\eta(\theta_k)$ would be close to a onehot-like vector but the variance of gradients would be large. When η is large, $\sigma_\eta(\theta_k)$ would be smoother but the variance of gradients would be small. Additionally, Bernoulli distribution, as a typical non-differentiable distribution, can be interpreted as a special binary case of Gumbel-softmax. Hence, Gumbel-softmax can also be served as an approximation to Bernoulli distributions, resulting in *Relaxed Bernoulli*, which can be expressed as:

$$\text{Ber}_{\text{relaxed}}(b|p, \eta) = \zeta \left[\frac{1}{\eta} \left(\log \frac{p}{1-p} + \log \frac{u}{1-u} \right) \right], \quad (2.13)$$

where $b \in \{0, 1\}$ is the resulting sample, $\zeta(x) = \frac{1}{1+\exp(-x)}$ is a sigmoid function, and u is the value sampled from the uniform distribution $U(0, 1)$. The distribution will be similar to Bernoulli with a lower temperature $\eta \rightarrow 0$.

Straight-Through Estimator (STE) [82] is a simple gradient estimator to bypass the non-differentiable operations $f(\cdot)$ by forcing the operator gradient to $\frac{df(x)}{dx} = 1$, where x refers to whatever input is fed into $f(\cdot)$ during the forward pass. Though not mathematically rigorous, STE maintains the gradient flow by approximating the function gradients as the identity function as if the non-differentiable operation was not existed during the backpropagation. Notably, though other curated gradient values were explored [86], [87], the gradient of STE is commonly set to 1 to avoid erroneously scale the gradient up or down as experimented

in [82]. In practice, this simple estimator is efficient to implement and worked well for estimating the gradients of data augmentation [28]. However, since the higher gradient variance might be introduced due to the discrepancies between the forward and backward pass as pointed out by [84], recent works are more inclined to use sampling-based gradient estimation methods like REINFORCE [88]–[90] and RELAX [26], [27].

2.3.2 Sampling-Based Methods

The sampling-based methods use randomized sampling and stochastic gradients to estimate gradients for non-differentiable functions. Both RELAX and REINFORCE produce unbiased gradients while REINFORCE has higher variance. A key difference is that REINFORCE is used for estimating gradients through non-differentiable reward functions (e.g. losses), while RELAX is used for gradients through non-differentiable operations.

REINFORCE [91] is known as a score-function estimator or a Monte Carlo policy gradient method. It computes the likelihood ratio taking the advantage of differentiable log-derivatives to estimate the gradients of the augmentation policy search. Some studies [88]–[90] utilized the REINFORCE algorithm (*a.k.a* Monte Carlo stochastic relaxation as in [88]) to estimate the gradients of non-differentiable augmentation policy search process as:

$$J = \mathcal{L}(\cdot, \tau) = \int_{\tau \in T} \mathcal{L}(\cdot|\tau) p_\theta(\tau) d\tau, \quad (2.14)$$

$$\nabla_\theta J \approx \frac{1}{N} \sum_{n=1}^N \mathcal{L}_n \nabla_\theta \log(p(\tau_n)), \quad (2.15)$$

where $p(\tau_n)$ represents the probability of the policy τ_n . Note that the REINFORCE estimator may provide high variance gradients.

RELAX [83] is similar to REINFORCE, that uses Monte Carlo sampling while a continuous relaxation to obtain unbiased and low-variance gradient estimation since the reparameterization tricks may introduce biased gradients [84], [92]. Given b is a discrete random variable by introducing a “relaxed” continuous variable z , RELAX estimator can be expressed as:

$$\begin{aligned} \nabla_\theta f &= [f(b) - c_\phi(\tilde{z})] \nabla_\theta \log \hat{\pi}(b|\theta) + \nabla_\theta c_\phi(z) - \nabla_\theta c_\phi(\tilde{z}), \\ b &= H(z), z \sim \hat{\pi}(z|\theta), \tilde{z} \sim \hat{\pi}(z|b, \theta), \end{aligned} \quad (2.16)$$

where a differentiable neural network c_ϕ is used as the surrogate function of the target non-differentiable operator $f(\cdot)$, that $c_\phi(\cdot)$ are encouraged to approximate $f(\cdot)$. Notably, RELAX requires a continuous, reparameterizable distribution $\hat{\pi}(z|\theta)$ (e.g. Gumbel-softmax, Relaxed Bernoulli) and a deterministic mapping $H(z)$ such that $H(z) = b \sim p(b|\theta)$ when $z \sim p(z|\theta)$, and it applies the control variate both at a relaxed input $z \sim \hat{\pi}(z|\theta)$, and a relaxed input conditioned on the discrete variable b , denoted $\tilde{z} \sim \hat{\pi}(z|b, \theta)$.

3 NEURAL AUGMENTATION NETWORKS

Gradient-based optimization has been used in many areas in image processing [93]–[95]. With the development of deep learning, an intuitive intention is to integrate the augmentation pipeline into neural network computation graphs, so as to enable gradient computation and optimization for

image transformation operations. Many applications (e.g. image restoration [23], colorization [24], [25]) optimized image processing parameters with stochastic gradient descent. Moreover, Wang and Perez [2] trailed a 5-layer CNN to perform augmentation without any traditional augmentation methods, that implies the potential of neural augmentation methods, that implies the potential of neural augmentation methods. Essentially, neural augmentation directly embed DDA into the model as a part of neural networks to take the advantage of the maintained gradient flows of differentiable image transformations. This section introduces *neural augmentation networks*, then it summarizes current studies as *static neural augmentation* and *optimizable neural augmentation* depending on the optimization state as illustrated in Fig. 8a and Fig.8b.

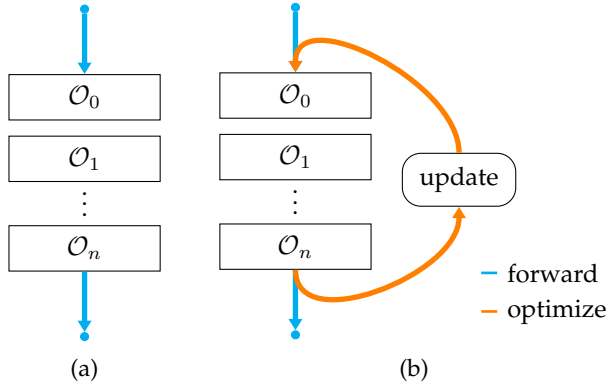


Fig. 8: Neural augmentation networks. (a) Static neural augmentation: composes differentiable data augmentation as neural network layers without optimizing augmentation parameters, (b) Optimizable neural augmentation: optimizes augmentation parameters in neural augmentation networks.

3.1 Neural Augmentation Networks

Earlier works such as spatial transform networks (STN) [41] integrated neural network layers to perform transformations to apply attention mechanism by performing spatial transformations. As shown in Fig. 9, a STN boils down to three components: a localization network, a grid generator, and a sampler.

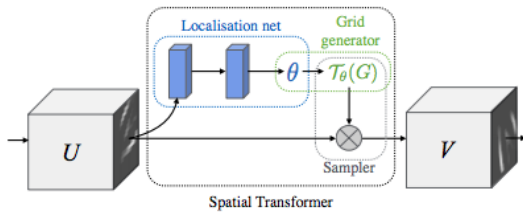


Fig. 9: An STN module [41] contains three parts: a) a **localization network** is a regular CNN which regresses the transformation parameters, b) a **grid generator** generates a grid of coordinates in the input image corresponding to each pixel from the output image, c) a **sampler** uses the parameters of the transformation and applies it to the input image.

More recently, OnlineAugment [96] used STN and variational auto-encoder (VAE) [97] networks to construct a series

augmentation network with an affine transformation network, a deformation network, and an intensity perturbation network. Gao *et al.* [98] followed the similar design with three convolutional neural networks to learn to generate transformations adaptively. Interestingly, the affine transformation network of OnlineAugment computed gradients w.r.t either generic feature maps or 1-D Guassian noise but not the images, as the authors indicated that the same spatial transformation shall be applicable to different images despite their contents. Tang *et al.* [96] and Gao *et al.* [98] characterized its data variations into three differentiable parts: affine transformation, local deformation, and appearance perturbations. Differently, TeachAugment [99] proposed differentiable neural augmentation models consisting a color augmentation model and a geometric augmentation model.

Meanwhile, apart from the routined image transformations (e.g. brightness, rotation adjustment), domain-specific image transformation were developed for the best fit for different domain areas. RenderGAN [100] defined four optimizable augmentation models including blurriness, lighting, background, and details to generate realistic data given labels, to analyse the complex social behavior of honey bees. Similarly, WaterGAN [31] defined an attenuation, a back scattering, and a camera model to generate underwater-like images from non-underwater images. Tseng *et al.* [30] leveraged *differentiable proxy models* including seven common image signal processors (ISP) stages to reproduce the entire ISP image transformation process.

3.1.1 Static Neural Augmentation Network

Static neural augmentation networks are essentially non-optimizable neural networks that allow gradients flow through augmentation layers without optimizing any parameters. For instance, Sharan *et al.* [101] employed a differentiable Gaussian filter layer as part of a UNet-based generator to improve the robustness of various tasks (e.g. landmark detection, semantic segmentation). Sun *et al.* [102] adopted differentiable Sobel filters to maintain the gradient information from edge detection to enhance the image edge clarity. Recent studies [103]–[105] applied data augmentation for the discriminator of GANs to avoid discriminator overfitting. Those GAN training techniques used a set of differentiable image transformations to augment training data to augment samples before fed into the discriminator, which significantly improved the data-efficiency for GAN training. Zhao *et al.* [106] further stated a better performance would be obtained by applying the exact transformation on both training data and generated samples at the same time. Additionally, Sablayrolles *et al.* [32] investigated data marking strategies to detect if a marked dataset has been used for trained models, which integrated DDA to trace the augmentation gradients. Furthermore, whilst training GANs, Tran *et al.* [105] performed a theoretically analysis that stated the preservation of Jensen-Shannon (JS) divergence between the input data and generated data can be guaranteed if differentiable and invertible³ (bijective) transformations are performed. With the development of

3. Invertible operations refer to the image operations that can fully restore the original images without information losses (e.g. rotation with 90, 180, and 270 degrees).

image synthesis methods, StableRep [60] directly employed diffusion models as static neural augmentation networks to generate positive image pairs by prompting the same text description for visual representation learning. Similarly, DIA [107] also used diffusion models as static neural augmentation networks to construct the negative counterpart of a given image for fine-grained representation learning.

3.1.2 Optimizable Neural Augmentation Network

Optimizable neural augmentation network optimizes appointed augmentation parameters via backpropagation. Typically, they are predefined differentiable neural augmentation models that can be integrated as a part of the network optimization graphs, where the gradient comes from either differentiable operators or gradient estimators.

An earlier work of neural augmentation is Smart Augmentation [108], which aims at learning suitable augmentations when training deep neural networks. Notably, augmentation optimization can lead to *augmentation avoidance*. As observed by Benton *et al.* [109], affine transformation parameters would tend to be optimized to not perform any augmentation with the standard cross-entropy loss. The authors hereby proposed to broaden the augmentation distributions with an extra regularization term $R(\mu) = -\|\mu\|_2$ to penalize the augmentation magnitude μ to avoid collapsing. Since the formula is for the single transformation, it neglected the other degrees of freedom of policy selection while a new hyperparameter could lead to *augmentation avoidance* again. Thus, Rommel *et al.* [110] added another selection weight ω to penalize the product of weights and magnitudes $R(\omega, \mu) = -\|\omega \odot \mu\|_2$ to avoid collapsing.

Inspired by adversarial training [3], another branch of studies [79], [111] tend to augment examples with adversarial methods. Unlike the direct end-to-end gradient descent optimization, adversarial examples are considered "harder" to learn with, which could lead to a better model generalization. However, adversarial training is unstable without any constraint due to the risks of collapsing the inherent meanings of images, resulting in out-of-distribution data. Ratner *et al.* [112] introduced the concept of *null class mapping*, representing the data points that have been destructively transformed without preserving the original labels. Formally, the *null class mapping* pitfall is given as:

$$m(\tau(x)) \in \{m(x), m_\phi\}, \quad (3.1)$$

where $m(\cdot)$ denotes the label mapping function for obtaining the ground truth of any given data, τ represents the augmentation, and m_ϕ represents an out-of-distribution null class. To avoid generating out-of-distribution example, the authors introduced a generative adversarial objective and diversity objective to minimize the probability of a generated sequence mapping augmented data $\tau(x)$ to m_ϕ . Meanwhile, different regularizers were proposed to regularize the transformations to avoid the null class mapping. VAT [79] is a method to perform adversarial training that regularized the generated examples with local distribution smoothness (LDS). RAT [111] further extended VAT by richer data transformations. OnlineAugment [96] added a regularization term to constrain the augmented data within reasonable distributions. TeachAugment [99] leveraged a

teacher model to guarantee the generated images are meaningful to relax the complexity of parameter tuning, without any prior knowledge.

3.2 Relation to Physical Informed Neural Networks

Another research trend focuses on physical informed neural networks (PINN) [113], which incorporate the prior-knowledge of physical principles into neural networks. Specifically, physical laws are interpreted in a differentiable manner as differentiable transformations. For example, Dubois *et al.* [114] utilized a Fourier space transformation matrix to transform 2D magnetisation to magnetic fields as follows:

$$\begin{bmatrix} \mathcal{B}_x \\ \mathcal{B}_y \\ \mathcal{B}_z \end{bmatrix} = \frac{-1}{\alpha} \begin{bmatrix} k_x^2/k & k_x k_y/k & i k_x \\ k_x k_y/k & k_y^2/k & i k_y \\ i k_x & i k_y & -k \end{bmatrix} \begin{bmatrix} \mathcal{V}_x \\ \mathcal{V}_y \\ \mathcal{V}_z \end{bmatrix}, \quad (3.2)$$

where $\mathcal{B}$ and $\mathcal{V}$ are the Fourier-transformed magnetic field and magnetisation vector respectively, k_x and k_y are the Fourier space coordinates, $k = \sqrt{k_x^2 + k_y^2}$, and the detailed approximation of α can be found in [114]. In a way, PINN can be seen as a specially constrained case of differentiable transformations or neural augmentation embedded with the particular prior knowledge.

4 DIFFERENTIABLE AUGMENTATION SEARCH

Among optimizable neural augmentation networks, differentiable automatic data augmentation is of particular interest. Automatic data augmentation refers to the process to find the best data augmentation policies for a specific dataset. As illustrated in Fig. 10a, it attempts to search through a predefined search space of augmentation policies to find optimal augmentation strategies, while the search space often contains a policy matrix that enumerated different compositions of augmentation strategies. Namely, with augmentation policy τ to apply a sequence of transformations for the augmented data $\tau(x)$, and a evaluation function f to evaluate the learnt policy by $f(\cdot)$, the general task of automatic data augmentation is to find the best augmentation policy τ_* for the best evaluation result as follows:

$$\tau_* = \arg \max_{\tau} f(\cdot | \tau(x)), \quad (4.1)$$

where the evaluation function f normally denotes the target neural network model under this context. In [35], a comprehensive survey on automatic data augmentation methods are delivered. This section introduces different objectives of automatic data augmentation, with a focus on the combination with differentiable methods. Benefited from DDA, one may optimize augmentation parameters directly for the optimal augmentation strategy with gradient-based optimization methods. Fig. 10 presents a visual demonstration regarding the typical automatic augmentation search methods used in the literature.

4.1 Objectives

The objective of obtaining the best augmentation policy, *a.k.a* policy optimization, can be described as a bilevel optimization problem, while some literatures surrogated the problem

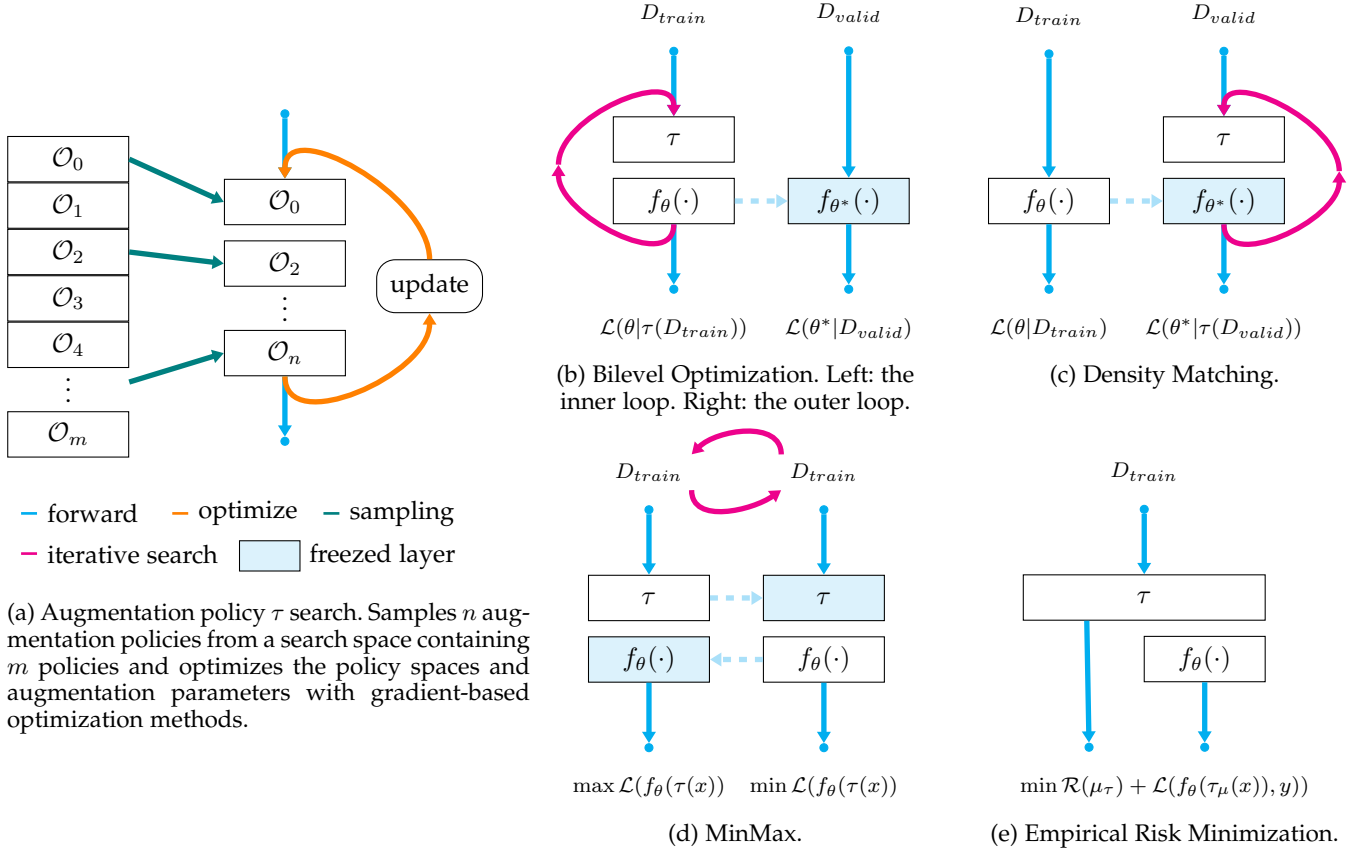


Fig. 10: Typical automatic augmentation search methods. (b) *Bilevel optimization* is a brutal-force method selects τ then trains f_{θ} multiple times, then select the combination that results in the lowest validation loss. (c) In *density match*, $f_{\theta}(\cdot)$ only needs to be trained for one time without augmenting the training data, then searching for the best τ to minimize the validation loss. (d) A *minmax* game aims to find out the augmentation policy τ to maximize the loss while to train the model f_{θ} to minimize the loss. (e) The *empirical risk minimization* methods normally add an additional regularization term to regularize the augmentation policy τ .

as matching problems to reduce the computation cost. For training neural neural networks, one common assumption is that hard examples could make training more effective and efficient [115]. Thus, some studies applied adversarial techniques with a focus on augmenting examples to hinder the learning. This section introduces four common optimizing objectives in the literature, including *bilevel optimization*, *matching*, *minmax*, and *empirical risk minimization*.

4.1.1 Bilevel Optimization Problem

Many augmentation policy searching methods [27], [88], [90], [116], [117] aim at solving a bilevel optimization problem [118], that the inner optimization task (lower-level) is embedded within the outer optimization task (upper-level). Commonly, the inner level is the *model weight optimization* that optimizes network parameters θ on training data using a given augmentation policy candidate τ , while the outer level is the *augmentation policy optimization* that optimizes the policy τ given the results of the inner level problem. More precisely, the bilevel optimization problem is formulated as:

$$\begin{aligned} \min_{\tau} \quad & \mathcal{L}(\theta^*|D_{valid}) \\ \text{s.t.} \quad & \theta^* \in \arg \min_{\theta} \mathcal{L}(\theta|\tau(D_{train})), \end{aligned} \quad (4.2)$$

where $\mathcal{L}(\theta|D)$ represents the loss over dataset D given θ .

Bilevel optimization is a typical split-optimization formulation that iteratively optimizes θ and τ respectively. It has gained its popularity since the work of AutoAugment [116], which consists of two components of a search algorithm and a search space. AutoAugment designed a policy search space with sub-policies, in which each sub-policy is parametrized into an image processing function, a probability, and a magnitude. It relies on the validation data to evaluate the augmentation performances while searching in the discrete policy space, resulting in inefficient bilevel optimizations. The complete bilevel optimization pipeline is computational expensive as stated in AutoAugment [116]. PBA [119] proposed to learn the optimal augmentation schedule rather than transformations. Tian *et al.* [117] proposed *augmentation-wise weight sharing* to pretrain the network weights θ with a shared augmentation before search the policy τ , that could significantly speed up the optimization. OHL-Auto-Aug [90] proposed an online bilevel optimization approach for improving the search efficiency and the final classification accuracy, which adopted the REINFORCE [91] estimator for approximating the gradients of the validation accuracy w.r.t augmentation parameters.

Inspired by DARTS [120], a differentiable neural architecture search framework, faster search can be achieved with

gradient-based optimization by relaxing the discrete search space to be continuous. DADA [26] and ADDA [27] relaxed the augmentation policy spaces to improve the searching efficiency to effectively obtain optimal policies with reduced computational complexity. Furthermore, DDAS [121] approximated the inner optimization with one-step gradient to avoid the training process. As pointed out by [96], offline searched policies are isolated from the downstream tasks, and they may not be domain-agnostic toward class-preserving, while the online augmentation search could mitigate the problems. DABO [122] adopted DDA to solve bilevel optimization problem in an online manner whilst training, while MADAO [29] proposed to solve the bilevel optimization problem by optimizing θ and τ simultaneously with Neumann series approximation in an online manner, that heavily shrunk the searching time from days to hours.

4.1.2 Matching Problem

Density Matching. Since the bilevel optimizations are computationally demanding, Fast AutoAugment [123] and Faster AutoAugment [28] considered augmentation policy searching as a *density matching* problem, that to match the density of a dataset D_1 with the density of augmented dataset D_2 , which works as a special case of *bilevel optimization*. In particular, *density matching* that sees data augmentation as a process that fills missing data points in training data. Fast AutoAugment [123] proposed to match the density between the training set D_{train} and the density of augmented validation set D_{val} . Specifically, the authors divided the training data set D_{train} to k subsets $\{D_{train}^1, \dots, D_{train}^k\}$ and each subset is split into two subsets D_M^n and D_A^n where $n = 1, \dots, k$. The subset D_M^n was used to learn the model parameters θ , while D_A^n was used for searching the augmentation policy τ . During the training, k models are firstly trained on D_M^n without any augmentation, then the model will be frozen to search for the best augmentation policy τ^k via Bayesian optimization techniques for each pair of D_M^n and D_A^n . Intuitively, it requires each D_M is a representative sample of D_{train} , then a policy that matches densities between augmented D_A and D_M will also match densities between augmented D_{train} and D_{valid} . With the expected model performance $R(\theta|D)$, *density matching* optimizes the following objective:

$$\tau_* = \arg \max_{\tau} R(\theta^* | \tau(D_A)), \quad (4.3)$$

where τ_* approximately minimizes the density distance between D_M and $\tau(D_A)$ by maximizing the model performance with the same parameter θ . The final τ_* is obtained by merging all the τ^n . More recently, Faster AutoAugment [28] solved *density matching* with DDA techniques, that directly minimized the Wasserstein distance [124] between D_M^n and $\tau^n(D_A^n)$ where τ^n is differentiable. Additionally, a classification loss is deployed for preserving image labels to avoid images being overly transformed:

$$l = \mathcal{L}(f(D_M), y_M) + \mathcal{L}(f(\tau(D_A)), y_A), \quad (4.4)$$

where f is the image classifier and $\mathcal{L}$ is the loss function. However, both methods adopt shallow augmentation layers (*i.e.* 2 sub-policies), since they are based on brute-force searching that prohibits the use of multiple sub-policies.

Gradient Matching. Deep AutoAugment [125] formulated the problem as a regularized *gradient matching* problem, which manages to steer the gradient of augmented training data towards the gradient of the validation batch by maximizing their cosine similarities. With $g(x, \mu)$ representing the gradients of augmentation where μ is the parameters of τ , and g_v being the gradients of validation data, gradient matching can be formulated as:

$$\tau_* = \arg \max_{\tau} \text{cosineSimilarity}(g_v, g(x, \mu)). \quad (4.5)$$

This method also tackled the problem of the search space's dimensionality exponentially increasing when more augmentation layers are added, which is achieved by progressively stacking augmentation layers, optimizing each layer on the data distribution transformed by the previous augmentation layers.

4.1.3 MinMax Adversarial Problem

The joint optimization of the target network parameters θ and augmentation policy network parameters τ is assumed to be superior since the augmentation networks are updated according to the learning state of the target network models [29], [89], [126]. Many adversarial learning-based methods play min-max games to perform augmentations to maximize the loss for the target model, to improve model generalization. Particularly, non-differentiable augmentation operations tend to be avoided to maintain the gradient flow from the target network to augmentation policy network. Earlier works performed adaptive transformation selection to maximize the end model loss [127], or to learn augmentation with adversarial objectives [112]. Adversarial AutoAugment [89] applied REINFORCE algorithm to relax the non-differentiable augmentation policy network to play min-max games towards generating adaptive augmentation policies in an online manner. It formulated the augmentation searching problem as a min-max game, that target network θ minimizes the training loss while the policy network τ maximizes the training loss. The generic formulation is:

$$\min_{\theta} \max_{\tau \in T} E_{x, y \sim D} [\mathcal{L}(f_{\theta}(\tau(x), y))], \quad (4.6)$$

where x and y are the image and label data sampled from the training set D , $f_{\theta}(\cdot)$ is the model parameterized by θ , $\mathcal{L}(\cdot)$ is the loss function, and S is all the available augmentations. Gao *et al.* [98] further combined adversarial training techniques and Adversarial AutoAugment to employ a regularized adversarial training framework with an additional min-max objectives, along with an augmentation magnitude regularization term to penalize large augmentation to avoid out-of-distribution data. OnlineAugment [96] utilized a meta-learned augmentation network along with a differentiable online data augmentation scheme based on adversarial training. To reduce the many regularization hyperparameters of OnlineAugment, TeachAugment [99] leveraged a teacher model to avoid careful parameter tuning. Additionally, UADA [126] applied adaptive adversarial data augmentation without a policy searching space to adaptively update augmentation parameters along with the directions of loss gradient (e.g. adversarial attack).

4.1.4 Empirical Risk Minimization Problem

Empirical risk minimization (ERM) is another joint optimization method that commonly offer an integral loss function to minimize the training loss while maximize the augmentation diversity. As introduced in Section 3.1.2, Smart Augmentation [108], Augerino [109], AugNet [110], and InstaAug [128] are the typical end-to-end learning methods based on differentiable augmentation operators, that minimize the empirical risk whilst maintaining a high variance of available augmentation without *augmentation avoidance*. Technically, with model parameter θ and augmentation parameter μ , for any image x and label y , the loss function of Augerino [109] is defined as:

$$\min_{\theta, \mu} [\mathbb{E}_{\tau \sim p_{\mu}(\tau)} [\mathcal{L}(f_{\theta}(\tau_{\mu}(x)), y)] + \lambda \mathbb{R}(\mu)], \quad (4.7)$$

where $\mathcal{L}$ is a loss function (e.g. cross-entropy) and $\lambda \mathbb{R}(\mu)$ is a regularization term that encourages large transformations where $\mathbb{R}(\mu) = -\|\mu\|_2$. AugNet [110] extended Augerino with hierarchical differentiable data augmentation layers and introduced the augmentation weights ω to select transformations which encode the strongest data invariance. Additionally, AugNet revised Augerino's regularization term $-\|\mu\|_2$ to $-\|\omega \odot \mu\|_2$ to prevent the potential *augmentation avoidance* from the introduced hyperparameter ω . Apart from learning global augmentations, InstaAug [128] proposed to perform instance-specific augmentation that aims at learning more diverse augmentations that also preserve labels compared to global augmentations. Meanwhile, GA3N [129] attempted to include GAN operations as additional policies into the policy space then applied adversarial training between policy and target networks to find optimal policies. The target network minimizes the loss on real and augmented samples, while the policy network maximizes the loss to generate harder augmented samples.

4.2 Differentiable Relaxation

Aforementioned methods mainly involved with a non-differentiable policy space and a number of predefined augmentation policies. This subsection highlights the differentiable relaxation techniques for those propositions.

4.2.1 Policy Space Relaxation

Subpolicies are scattered in the search space in a discrete manner, that posing optimization difficulties due to the discontinuity. DARTS [120] is a gradient-based neural architecture search method to solve the bi-level optimization problem. Essentially, it relaxes the categorical selection of a particular operation to a softmax over all possible operations. Similarly, most of the studies adopted this approach to relax the discrete space into a continuous one. Specifically, Faster AutoAugment [28] utilized a softmax function $\sigma(z) = \frac{\exp z/\eta}{\sum_j \exp z_j/\eta}$ with temperature η , so that $\sigma(z)$ would be a onehot-like vector when η is small. MADAO [29] replaced the softmax function with the Gumbel-softmax ((2.12)) to improve the differentiability. DADA [26] and ADDA [27] pointed the Gumbel-softmax gradients are biased, so that the search efficiency can be further improved by adding an unbiased RELAX gradient estimator [83]. Meanwhile, this

relaxation can be used to sample the whole subpolicies than operations within subpolicies as in DADA [26].

Notably, although the relaxation shortened improved the searching efficiency, some practitioners claim it would introduce discrepancies between the real and relaxed augmentation spaces [125]. Thus, instead of relaxation, Deep AutoAugment [125] proposed neural policy layers and each layer is a 139-dimensional categorical distribution to match the 139 $\{\text{operation}, \text{magnitude}\}$ pairs. GA3N [129] modeled the search space using a LSTM network that consequentially selecting operations and magnitudes. Additionally, DDAS [121] directly applied the one-step gradient update approach with the expectation of training loss instead of reparameterization tricks or gradient estimators.

4.2.2 Operation Relaxation

Most augmentation searching methods [116], [119] treated image augmentation as a gradient-free preprocessor, that is not backpropagatable through the network computation graphs. In order to take the advantage of the gradient-based optimization, DADA [26] and CADDA [27] utilized RELAX algorithm to approximate gradients for all the augmentation operations, neglected the operations that are naturally differentiable. Hence, Hataya *et al.* [28], [29] integrated DDA to avoid inaccurate gradient approximations, while adopted STE to approximate non-differentiable operations. The integrated DDA operation $O(\cdot; p, \mu)$ would be differentiable w.r.t. the probability p and the magnitude μ as:

$$\hat{x} = b \cdot O(x|\mu_O) + (1 - b) \cdot x, \quad b \sim \text{Ber}_{\text{relaxed}}(b|p_O), \quad (4.8)$$

where $\text{Ber}_{\text{relaxed}}$ relaxes each operation to be differentiable w.r.t. the probability p and the gradient of the resulting image is solved by a linear addition. Notably, the differentiable operations can naturally be differentiated w.r.t. the magnitude μ , and STE is mostly applied for the non-differentiable operations under DDA settings [26]–[29].

5 DISCUSSION AND FUTURE OPPORTUNITIES

This section discusses the challenges and potential opportunities in the area of DDA. In Table 2, we provide a summary of the DDA methods existing in the literature. Furthermore, we discuss the potential research directions from three perspectives: *modeling*, *optimization*, and *evaluation*.

5.1 Modeling

Differentiable Operations: Many non-differentiable operations exist referring to Table 1. In order to avoid the inaccurate approximations from gradient estimators [121], [132], differentiable modelings of non-differentiable operations are preferred. For example, as indicated earlier, the modeling of differentiable *equalize* might be possible with the aid of differentiable histogram calculation methods [47]–[49]. Meanwhile, some previous works [26]–[29] optimize probabilities p and magnitudes μ , while some studies [96], [98], [99] optimized augmentation networks instead. In addition to develop differentiable operations, another direction is to investigate the DDA parameterization strategies.

Class-Dependent Augmentation: Balestrieri *et al.* [133] demonstrated the class dependent regularization effects for

TABLE 2: A summary of DDA literature. By the types of neural augmentation layers, we categorize current works to static (i.e. static neural augmentation) and optimizable (i.e. optimizable neural augmentation). We also summarize operation gradient sources to *direct* if the gradients are directly computed by DDA operations, *network* if the gradients are computed from augmentation networks, along with the corresponding gradient estimators if used. We categorize the search space to *routined* if a predefined augmentation pipeline is used, to *n/a* if not a searching method, and to *relaxed* if a relaxed policy space is available.

Name	venue	op. type	op. gradient	task	search objective	search space
Augerino [109]	NeurIPS 2020	optimizable	network	classification	ERM	routined
DiffAugment [104]	NeurIPS 2020	static	direct	synthesis	n/a	n/a
Karras <i>et al.</i> [103]	NeurIPS 2020	static	direct	synthesis	n/a	n/a
Sablayrolles <i>et al.</i> [32]	ICML 2020	static	direct	data marking	n/a	n/a
Faster AutoAugment [28]	ECCV 2020	optimizable	direct, STE	classification	density match	relaxed
OnlineAugment [96]	ECCV 2020	optimizable	network	classification	min-max	routined
DADA [26]	ECCV 2020	optimizable	direct, RELAX	classification	bilevel	relaxed
DABO [122]	WACV 2021	optimizable	direct	classification	bilevel	routined
Gao <i>et al.</i> [98]	IPMI 2021	optimizable	network	segmentation	min-max	routined
CADDA [27]	ICLR 2021	optimizable	direct, RELAX	classification	bilevel	relaxed
AdaAug [130]	ICLR 2021	optimizable	direct, STE	classification	density match	relaxed
Tran <i>et al.</i> [105]	TIP 2021	static	direct	synthesis	n/a	n/a
Sharan <i>et al.</i> [101]	JBHI 2021	static	direct	synthesis, detection	n/a	n/a
TeachAugment [99]	CVPR 2022	optimizable	network	classification	min-max	routined
AugNet [110]	NeurIPS 2022	optimizable	network	classification	ERM	routined
MADAO [29]	WACV 2022	optimizable	direct, STE	classification	bilevel	relaxed
GA3N [129]	PR 2022	optimizable	network	classification	ERM	relaxed
InstaAug [128]	ICML 2023	optimizable	network	classification	ERM	routined
DRA [131]	TIP 2023	optimizable	direct, STE	classification	bilevel	relaxed

data augmentations, which creates class-dependent model biases. For example, in MNIST dataset, we aim at obtaining a model to be invariant to rotations of a ‘6’ up until it looks more like a ‘9’ and vice versa [109], while the invariant rotation interval differs for ‘8’. Similar situation applied to image instances. Apparently, the class-dependent or even instance-dependent data augmentation can hardly be manually designed. While differentiable augmentations fit into this problem, Rommel *et al.* [27] proposed a class-dependent augmentation for 1D EEG signals with DDA techniques while Miao *et al.* [128] presented an instance-dependent differentiable image augmentation method. A further modeling of class-dependent or instance-dependent image data augmentation might be promising to reduce the class/instance-dependent biases.

Applications: Current research trends of DDA are mostly towards finding the best augmentation policies to improve the model performances for downstream tasks. Nevertheless, previous studies stated that traditional data augmentation methods are proven to be effective for other tasks, such as the passive defences of adversarial attacks [134]–[136] and mitigating the membership inference attacks towards differential privacy [137]. A recent study [32] integrated DDA to trace the training dataset. Plus, the curated differentiable transformations also contributed to PINN applications [114]. It would be interesting to see more DDA applications such as integrating DDA into adversarial defenses and membership inference defences.

5.2 Optimization

Robust Gradient Estimation: As aforementioned, gradient estimation is yet effective and practical in many DDA applications. Despite the introduced bias and variance due to different gradient estimators [83], the approximation itself might be problematic. For example, some studies [28], [29]

directly adopted DDA and applied STE to bypass the non-differentiable augmentations, which clearly introduced extra biases during the optimization. Meanwhile, DADA [26] and ADDA [27] used unbiased RELAX to replace the biased Gumbel-softmax for a better performances. However, some researches [121], [132] have further pointed out that the estimated second-order gradients of DADA may lead to an inaccurate gradient direction. A recent research [138] also dived into the automatic selection of gradient estimators. Despite developing more robust gradient estimation algorithms, another research direction can be the automatic gradient estimator selection algorithms.

Variance Regularization: Limited invariances are provided in typical augmentation methods, so that model-based augmentation is preferred in terms of a larger invariance space [62], while Ravuri *et al.* [139] claimed that generative augmentation method may fail to improve model performances. Thus, the more restricted neural augmentation might come as an aid with curated optimizable transformation routines. As discussed in Section 3.1.2, DDA optimization might fall into the two extremes of *augmentation avoidance* and *null class*. In this case, several problems need to be addressed, such as i) regularize the augmentation variance to perform *label-preserving transformations* and ii) regularize the post-transform label variance (e.g. the loss computation in MixUp [4]) to maintain meaningful supervision.

5.3 Evaluation

Gradient-Enhanced Theoretical Analysis: Apart from the studies (e.g. [101], [102], [104]) that utilized DDA statically, optimizing augmentation parameters remains critical in many applications, hereby many objectives and regularizers [109], [110] are proposed to regularize the optimization process. However, with an aim of improving the validation set performance, a theoretical analysis is needed to explain

the optimized augmentation policies. Dao *et al.* [1] carried out a kernel alignment metric to analyze whether a transformation would improve the generalization performance without performing end-to-end training. Additionally, Wu *et al.* [140] presented an uncertainty-based theoretical analysis to demonstrate the rationale under various augmentations, regarding the abilities of different transformation methods for reducing estimation errors. As the gradient-based formulation in [125], a gradient-enhanced theoretical analyzing framework might assist explaining the optimization rationale while yet challenging.

6 CONCLUSION

In this survey, we aimed at presenting a structured yet extensive overview of the current research directions on differential data augmentation applied to image processing, which could effectively improve the model performances in many downstream applications including object detection, segmentation, GANs, and automatic augmentations. We firstly introduced the key elements (including concept, basic operations, and gradient approximations) of DDA, then we summarized current application paradigms, along with relevant approaches and works that we categorized by the types of neural augmentation layers, specifically static and optimizable neural augmentation. Finally, we discussed the current challenges and highlighted the future potentials from three perspectives of modeling, optimization, and evaluation. Data augmentation plays an important role in dealing with overfitting and modern contrastive learning. Differentiable data augmentation, as a trending area, has already demonstrated its potential in data-efficient learning, dataset tracking, and efficient augmentation policy searching. With this survey, we provided a guide for researchers to better grasp the rationale behind differentiable data augmentation and to inspire their interest in further researches in this field.

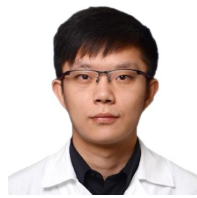
REFERENCES

- [1] T. Dao, A. Gu, A. Ratner, V. Smith, C. De Sa, and C. Ré, “A kernel theory of modern data augmentation,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 1528–1537.
- [2] L. Perez and J. Wang, “The effectiveness of data augmentation in image classification using deep learning,” *arXiv preprint arXiv:1712.04621*, 2017.
- [3] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards deep learning models resistant to adversarial attacks,” *arXiv preprint arXiv:1706.06083*, 2017.
- [4] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond empirical risk minimization,” *arXiv preprint arXiv:1710.09412*, 2017.
- [5] S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “Cutmix: Regularization strategy to train strong classifiers with localizable features,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 6023–6032.
- [6] X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel, “Infogan: Interpretable representation learning by information maximizing generative adversarial nets,” *Advances in neural information processing systems*, vol. 29, 2016.
- [7] G. Ning, G. Chen, C. Tan, S. Luo, L. Bo, and H. Huang, “Data augmentation for object detection via differentiable neural rendering,” *arXiv preprint arXiv:2103.02852*, 2021.
- [8] V. Verma, A. Lamb, C. Beckham, A. Najafi, I. Mitliagkas, D. Lopez-Paz, and Y. Bengio, “Manifold mixup: Better representations by interpolating hidden states,” in *International Conference on Machine Learning*. PMLR, 2019, pp. 6438–6447.
- [9] T.-H. Cheung and D.-Y. Yeung, “Modals: Modality-agnostic automated data augmentation in the latent space,” in *International Conference on Learning Representations*, 2020.
- [10] A. Sinha, K. Ayush, J. Song, B. Uzkent, H. Jin, and S. Ermon, “Negative data augmentation,” *arXiv preprint arXiv:2102.05113*, 2021.
- [11] D. Berthelot, N. Carlini, I. Goodfellow, N. Papernot, A. Oliver, and C. A. Raffel, “Mixmatch: A holistic approach to semi-supervised learning,” *Advances in neural information processing systems*, vol. 32, 2019.
- [12] D. Berthelot, N. Carlini, E. D. Cubuk, A. Kurakin, K. Sohn, H. Zhang, and C. Raffel, “Remixmatch: Semi-supervised learning with distribution alignment and augmentation anchoring,” *arXiv preprint arXiv:1911.09785*, 2019.
- [13] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, “Fixmatch: Simplifying semi-supervised learning with consistency and confidence,” *Advances in neural information processing systems*, vol. 33, pp. 596–608, 2020.
- [14] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International conference on machine learning*. PMLR, 2020, pp. 1597–1607.
- [15] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 9729–9738.
- [16] J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar *et al.*, “Bootstrap your own latent—a new approach to self-supervised learning,” *Advances in neural information processing systems*, vol. 33, pp. 21 271–21 284, 2020.
- [17] X. Chen and K. He, “Exploring simple siamese representation learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 15 750–15 758.
- [18] E. Riba, D. Mishkin, D. Ponsa, E. Rublee, and G. Bradski, “Kornia: an open source differentiable computer vision library for pytorch,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2020, pp. 3674–3683.
- [19] T.-M. Li, M. Gharbi, A. Adams, F. Durand, and J. Ragan-Kelley, “Differentiable programming for image processing and deep learning in halide,” *ACM Transactions on Graphics (ToG)*, vol. 37, no. 4, pp. 1–13, 2018.
- [20] K. M. Yi, Y. Verdie, P. Fua, and V. Lepetit, “Learning to assign orientations to feature points,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 107–116.
- [21] C. Bowles, L. Chen, R. Guerrero, P. Bentley, R. Gunn, A. Hammers, D. A. Dickie, M. V. Hernández, J. Wardlaw, and D. Rueckert, “Gan augmentation: Augmenting training data using generative adversarial networks,” *arXiv preprint arXiv:1810.10863*, 2018.
- [22] O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin, and J. Matas, “Deblurgan: Blind motion deblurring using conditional adversarial networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 8183–8192.
- [23] M. Gharbi, J. Chen, J. T. Barron, S. W. Hasinoff, and F. Durand, “Deep bilateral learning for real-time image enhancement,” *ACM Transactions on Graphics (TOG)*, vol. 36, no. 4, pp. 1–12, 2017.
- [24] S. Iizuka, E. Simo-Serra, and H. Ishikawa, “Let there be color! joint end-to-end learning of global and local image priors for automatic image colorization with simultaneous classification,” *ACM Transactions on Graphics (ToG)*, vol. 35, no. 4, pp. 1–11, 2016.
- [25] R. Zhang, P. Isola, and A. A. Efros, “Colorful image colorization,” in *European conference on computer vision*. Springer, 2016, pp. 649–666.
- [26] Y. Li, G. Hu, Y. Wang, T. Hospedales, N. M. Robertson, and Y. Yang, “Dada: Differentiable automatic data augmentation,” *arXiv preprint arXiv:2003.03780*, 2020.
- [27] C. Rommel, T. Moreau, J. Paillard, and A. Gramfort, “Cadda: Class-wise automatic differentiable data augmentation for eeg signals,” in *Proceedings of the 9th International Conference on Learning Representations*, 2021.
- [28] R. Hataya, J. Zdenek, K. Yoshizoe, and H. Nakayama, “Faster autoaugment: Learning augmentation strategies using backpropagation,” in *European Conference on Computer Vision*. Springer, 2020, pp. 1–16.
- [29] —, “Meta approach to data augmentation optimization,” in

- 2022 *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2022, pp. 3535–3544.
- [30] E. Tseng, F. Yu, Y. Yang, F. Mannan, K. S. Arnaud, D. Nowrouzezahrai, J.-F. Lalonde, and F. Heide, “Hyperparameter optimization in black-box image processing using differentiable proxies,” *ACM Trans. Graph.*, vol. 38, no. 4, jul 2019. [Online]. Available: <https://doi.org/10.1145/3306346.3322996>
- [31] J. Li, K. A. Skinner, R. M. Eustice, and M. Johnson-Roberson, “Watergan: Unsupervised generative network to enable real-time color correction of monocular underwater images,” *IEEE Robotics and Automation letters*, vol. 3, no. 1, pp. 387–394, 2017.
- [32] A. Sablayrolles, M. Douze, C. Schmid, and H. Jégou, “Radioactive data: tracing through training,” in *International Conference on Machine Learning*. PMLR, 2020, pp. 8326–8335.
- [33] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of big data*, vol. 6, no. 1, pp. 1–48, 2019.
- [34] M. Xu, S. Yoon, A. Fuentes, and D. S. Park, “A comprehensive survey of image augmentation techniques for deep learning,” *arXiv preprint arXiv:2205.01491*, 2022.
- [35] Z. Yang, R. O. Sinnott, J. Bailey, and Q. Ke, “A survey of automated data augmentation algorithms for deep learning-based image classification tasks,” *arXiv preprint arXiv:2206.06544*, 2022.
- [36] H. Naveed, “Survey: Image mixing and deleting for data augmentation,” *arXiv preprint arXiv:2106.07085*, 2021.
- [37] J. Schulman, N. Heess, T. Weber, and P. Abbeel, “Gradient estimation using stochastic computation graphs,” *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [38] W. Chen, J. Gao, H. Ling, E. Smith, J. Lehtinen, A. Jacobson, and S. Fidler, “Learning to predict 3d objects with an interpolation-based differentiable renderer,” in *Advances In Neural Information Processing Systems*, 2019.
- [39] O. Michel, R. Bar-On, R. Liu, S. Benaim, and R. Hanocka, “Text2mesh: Text-driven neural stylization for meshes,” *arXiv preprint arXiv:2112.03221*, 2021.
- [40] N. Passalis, S. Pedrazzi, R. Babuska, W. Burgard, D. Dias, F. Ferro, M. Gabbouj, O. Green, A. Iosifidis, E. Kayacan, J. Kober, O. Michel, N. Nikolaidis, P. Nousi, R. Pieters, M. Tzelepi, A. Valada, and A. Tefas, “Opendr: An open toolkit for enabling high performance, low footprint deep learning for robotics,” *arXiv preprint arXiv:2203.00403*, 2022.
- [41] M. Jaderberg, K. Simonyan, A. Zisserman *et al.*, “Spatial transformer networks,” *Advances in neural information processing systems*, vol. 28, 2015.
- [42] J. Shi, E. Riba, D. Mishkin, F. Moreno, and A. Nicolaou, “Differentiable data augmentation with kornia,” *arXiv preprint arXiv:2011.09832*, 2020.
- [43] P. Simard, D. Steinkraus, and J. Platt, “Best practices for convolutional neural networks applied to visual document analysis,” in *Seventh International Conference on Document Analysis and Recognition*, 2003. *Proceedings.*, 2003, pp. 958–963.
- [44] J. Albers, A. Svetlove, J. Alves, A. Kraupner, F. Di Lillo, A. Markus, G. Tromba, F. Alves, and C. Dullin, “Elastic transformation of histological slices allows precise co-registration with microct data sets for a refined virtual histology approach,” *Scientific Reports*, vol. 11, 05 2021.
- [45] N. Aldoj, F. Biavati, F. Michallek, S. Stober, and M. Dewey, “Automatic prostate and prostate zones segmentation of magnetic resonance images using densenet-like u-net,” *Scientific Reports*, vol. 10, 08 2020.
- [46] R. Szeliski, *Computer vision: algorithms and applications*. Springer Science & Business Media, 2010.
- [47] M. Avi-Aharon, A. Arbelle, and T. R. Raviv, “Deephist: Differentiable joint and color histogram layers for image-to-image translation,” *arXiv preprint arXiv:2005.03995*, 2020.
- [48] E. Ustinova and V. Lempitsky, “Learning deep embeddings with histogram loss,” *Advances in Neural Information Processing Systems*, vol. 29, 2016.
- [49] M. Avi-Aharon, A. Arbelle, and T. R. Raviv, “Differentiable histogram loss functions for intensity-based image-to-image translation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [50] D. Lewy and J. Mańdziuk, “An overview of mixing augmentation methods and augmentation strategies,” *Artificial Intelligence Review*, pp. 1–59, 2022.
- [51] H. Inoue, “Data augmentation by pairing samples for images classification,” *arXiv preprint arXiv:1801.02929*, 2018.
- [52] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “Yolov4: Optimal speed and accuracy of object detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [53] T. DeVries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout,” *arXiv preprint arXiv:1708.04552*, 2017.
- [54] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, “Random erasing data augmentation,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 07, 2020, pp. 13 001–13 008.
- [55] R. G. Lopes, D. Yin, B. Poole, J. Gilmer, and E. D. Cubuk, “Improving robustness without sacrificing accuracy with patch gaussian augmentation,” *arXiv preprint arXiv:1906.02611*, 2019.
- [56] D. Li, J. Yang, K. Kreis, A. Torralba, and S. Fidler, “Semantic segmentation with generative models: Semi-supervised learning and strong out-of-domain generalization,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 8300–8311.
- [57] S. Goyal, S.-A. Rebuffi, O. Wiles, F. Stimberg, D. A. Calian, and T. A. Mann, “Improving robustness using generated data,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 4218–4233, 2021.
- [58] S. Azizi, S. Kornblith, C. Saharia, M. Norouzi, and D. J. Fleet, “Synthetic data from diffusion models improves imagenet classification,” *arXiv preprint arXiv:2304.08466*, 2023.
- [59] D. Li, H. Ling, S. W. Kim, K. Kreis, S. Fidler, and A. Torralba, “Big-datasetgan: Synthesizing imagenet with pixel-wise annotations,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 21 330–21 340.
- [60] Y. Tian, L. Fan, P. Isola, H. Chang, and D. Krishnan, “Stablerep: Synthetic images from text-to-image models make strong visual representation learners,” *arXiv preprint arXiv:2306.00984*, 2023.
- [61] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” *arXiv preprint arXiv:1411.1784*, 2014.
- [62] A. Antoniou, A. Storkey, and H. Edwards, “Data augmentation generative adversarial networks,” *arXiv preprint arXiv:1711.04340*, 2017.
- [63] Y. Wang, A. Gonzalez-Garcia, D. Berga, L. Herranz, F. S. Khan, and J. v. d. Weijer, “Minegan: effective knowledge transfer from gans to target domains with few images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 9332–9341.
- [64] D. Bau, J.-Y. Zhu, J. Wulff, W. Peebles, H. Strobelt, B. Zhou, and A. Torralba, “Seeing what a gan cannot generate,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 4502–4511.
- [65] J.-Y. Zhu, P. Krähenbühl, E. Shechtman, and A. A. Efros, “Generative visual manipulation on the natural image manifold,” in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part V 14*. Springer, 2016, pp. 597–613.
- [66] W. Xia, Y. Zhang, Y. Yang, J.-H. Xue, B. Zhou, and M.-H. Yang, “Gan inversion: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 3, pp. 3121–3138, 2022.
- [67] D. T. Nguyen, C. T. Tran, T. T. Nguyen, C. B. Hoang, B. N. Nguyen, P. I. Cheong *et al.*, “Data augmentation for small face datasets and face verification by generative adversarial networks inversion,” in *2021 13th International Conference on Knowledge and Systems Engineering (KSE)*. IEEE, 2021, pp. 1–6.
- [68] M. Golhar, T. L. Bobrow, S. Ngamruengphong, and N. J. Durr, “Gan inversion for data augmentation to improve colonoscopy lesion classification,” *arXiv preprint arXiv:2205.02840*, 2022.
- [69] D. Epstein, T. Park, R. Zhang, E. Shechtman, and A. A. Efros, “Blobgan: Spatially disentangled scene representations,” in *European Conference on Computer Vision*. Springer, 2022, pp. 616–635.
- [70] B. Kavar, S. Zada, O. Lang, O. Tov, H. Chang, T. Dekel, I. Mosseri, and M. Irani, “Imagic: Text-based real image editing with diffusion models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 6007–6017.
- [71] L. Tsaban and A. Passos, “Ledits: Real image editing with ddpm inversion and semantic guidance,” *arXiv preprint arXiv:2307.00522*, 2023.
- [72] Y. Song, X. Shao, K. Chen, W. Zhang, Z. Jing, and M. Li, “Clipvg: Text-guided image manipulation using differentiable vector graphics,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 2, 2023, pp. 2312–2320.

- [73] J. Zhu, Y. Shen, D. Zhao, and B. Zhou, "In-domain gan inversion for real image editing," in *Proceedings of European Conference on Computer Vision (ECCV)*, 2020.
- [74] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.
- [75] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [76] A. Shafahi, M. Najibi, M. A. Ghiasi, Z. Xu, J. Dickerson, C. Studer, L. S. Davis, G. Taylor, and T. Goldstein, "Adversarial training for free!" *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [77] Y. Wang, D. Zou, J. Yi, J. Bailey, X. Ma, and Q. Gu, "Improving adversarial robustness requires revisiting misclassified examples," in *International conference on learning representations*, 2019.
- [78] C. Xie, Y. Wu, L. v. d. Maaten, A. L. Yuille, and K. He, "Feature denoising for improving adversarial robustness," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 501–509.
- [79] T. Miyato, S.-i. Maeda, M. Koyama, and S. Ishii, "Virtual adversarial training: a regularization method for supervised and semi-supervised learning," *IEEE transactions on pattern analysis and machine intelligence*, vol. 41, no. 8, pp. 1979–1993, 2018.
- [80] A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay, and D. Mukhopadhyay, "Adversarial attacks and defences: A survey," *arXiv preprint arXiv:1810.00069*, 2018.
- [81] Z. Qian, K. Huang, Q.-F. Wang, and X.-Y. Zhang, "A survey of robust adversarial training in pattern recognition: Fundamental, theory, and methodologies," *Pattern Recognition*, vol. 131, p. 108889, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320322003703>
- [82] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," *arXiv preprint arXiv:1308.3432*, 2013.
- [83] W. Grathwohl, D. Choi, Y. Wu, G. Roeder, and D. Duvenaud, "Backpropagation through the void: Optimizing control variates for black-box gradient estimation," in *Proceedings of the 5th International Conference on Learning Representations*, 2017.
- [84] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," *arXiv preprint arXiv:1611.01144*, 2016.
- [85] C. J. Maddison, D. Tarlow, and T. Minka, "A* sampling," in *Advances in Neural Information Processing Systems*, Z. Ghahramani, M. Welling, C. Cortes, N. Lawrence, and K. Weinberger, Eds., vol. 27. Curran Associates, Inc., 2014.
- [86] J. Chung, S. Ahn, and Y. Bengio, "Hierarchical multiscale recurrent neural networks," *arXiv preprint arXiv:1609.01704*, 2016.
- [87] P. Yin, J. Lyu, S. Zhang, S. Osher, Y. Qi, and J. Xin, "Understanding straight-through estimator in training activation quantized neural nets," *arXiv preprint arXiv:1903.05662*, 2019.
- [88] J. Xu, M. Li, and Z. Zhu, "Automatic data augmentation for 3d medical image segmentation," in *International Conference on Medical Image Computing and Computer-Assisted Intervention*. Springer, 2020, pp. 378–387.
- [89] X. Zhang, Q. Wang, J. Zhang, and Z. Zhong, "Adversarial autoaugment," *arXiv preprint arXiv:1912.11188*, 2019.
- [90] C. Lin, M. Guo, C. Li, X. Yuan, W. Wu, J. Yan, D. Lin, and W. Ouyang, "Online hyper-parameter learning for auto-augmentation strategy," in *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 6578–6587.
- [91] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, May 1992. [Online]. Available: <https://doi.org/10.1007/bf00992696>
- [92] C. J. Maddison, A. Mnih, and Y. W. Teh, "The concrete distribution: A continuous relaxation of discrete random variables," *arXiv preprint arXiv:1611.00712*, 2016.
- [93] L. I. Rudin, S. Osher, and E. Fatemi, "Nonlinear total variation based noise removal algorithms," *Physica D: nonlinear phenomena*, vol. 60, no. 1–4, pp. 259–268, 1992.
- [94] B. K. Horn and B. G. Schunck, "Determining optical flow," *Artificial intelligence*, vol. 17, no. 1–3, pp. 185–203, 1981.
- [95] S. Roth and M. J. Black, "Fields of experts: A framework for learning image priors," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, vol. 2. IEEE, 2005, pp. 860–867.
- [96] Z. Tang, Y. Gao, L. Karlinsky, P. Sattigeri, R. Feris, and D. Metaxas, "Onlineaugment: Online data augmentation with less domain knowledge," *arXiv preprint arXiv:2007.09271*, 2020.
- [97] D. P. Kingma and M. Welling, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [98] Y. Gao, Z. Tang, M. Zhou, and D. Metaxas, "Enabling data diversity: efficient automatic augmentation via regularized adversarial training," in *International Conference on Information Processing in Medical Imaging*. Springer, 2021, pp. 85–97.
- [99] T. Suzuki, "Teachaugment: Data augmentation optimization using teacher knowledge," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10904–10914.
- [100] L. Sixt, B. Wild, and T. Landgraf, "Rendergan: Generating realistic labeled data," *Frontiers in Robotics and AI*, vol. 5, p. 66, 2018.
- [101] L. Sharan, G. Romano, S. Koehler, H. Kelm, M. Karck, R. De Simone, and S. Engelhardt, "Mutually improved endoscopic image synthesis and landmark detection in unpaired image-to-image translation," *IEEE Journal of Biomedical and Health Informatics*, vol. 26, no. 1, pp. 127–138, 2021.
- [102] K. Sun, F. Meng, and Y. Tian, "Multi-level wavelet-based network embedded with edge enhancement information for underwater image enhancement," *Journal of Marine Science and Engineering*, vol. 10, no. 7, 2022. [Online]. Available: <https://www.mdpi.com/2077-1312/10/7/884>
- [103] T. Karras, M. Aittala, J. Hellsten, S. Laine, J. Lehtinen, and T. Aila, "Training generative adversarial networks with limited data," *Advances in Neural Information Processing Systems*, vol. 33, pp. 12 104–12 114, 2020.
- [104] S. Zhao, Z. Liu, J. Lin, J.-Y. Zhu, and S. Han, "Differentiable augmentation for data-efficient gan training," in *Conference on Neural Information Processing Systems (NeurIPS)*, 2020.
- [105] N.-T. Tran, V.-H. Tran, N.-B. Nguyen, T.-K. Nguyen, and N.-M. Cheung, "On data augmentation for gan training," *IEEE Transactions on Image Processing*, vol. 30, pp. 1882–1897, 2021.
- [106] B. Zhao and H. Bilen, "Dataset condensation with differentiable siamese augmentation," in *International Conference on Machine Learning*. PMLR, 2021, pp. 12 674–12 685.
- [107] J. Shi, P. Zhang, N. Zhang, H. Ghazzai, and Y. Massoud, "Dissolving is amplifying: Towards fine-grained anomaly detection," *arXiv preprint arXiv:2302.14696*, 2023.
- [108] J. Lemley, S. Bazrafkan, and P. Corcoran, "Smart augmentation learning an optimal data augmentation strategy," *Ieee Access*, vol. 5, pp. 5858–5869, 2017.
- [109] G. Benton, M. Finzi, P. Izmailov, and A. G. Wilson, "Learning invariances in neural networks from training data," *Advances in neural information processing systems*, vol. 33, pp. 17 605–17 616, 2020.
- [110] C. Rommel, T. Moreau, and A. Gramfort, "Deep invariant networks with differentiable augmentation layers," *arXiv preprint arXiv:2202.02142*, 2022.
- [111] T. Suzuki and I. Sato, "Adversarial transformations for semi-supervised learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 5916–5923.
- [112] A. J. Ratner, H. Ehrenberg, Z. Hussain, J. Dunnmon, and C. Ré, "Learning to compose domain-specific transformations for data augmentation," *Advances in neural information processing systems*, vol. 30, 2017.
- [113] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0021999118307125>
- [114] A. Dubois, D. Broadway, A. Stark, M. Tschudin, A. Healey, S. Huber, J.-P. Tetienne, E. Greplova, and P. Maletinsky, "Untrained physically informed neural network for image reconstruction of magnetic field sources," *arXiv preprint arXiv:2207.13720*, 2022.
- [115] A. Shrivastava, A. Gupta, and R. Girshick, "Training region-based object detectors with online hard example mining," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [116] E. D. Cubuk, B. Zoph, D. Mane, V. Vasudevan, and Q. V. Le, "Autoaugment: Learning augmentation strategies from data," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 113–123.

- [117] K. Tian, C. Lin, M. Sun, L. Zhou, J. Yan, and W. Ouyang, "Improving auto-augment via augmentation-wise weight sharing," *Advances in Neural Information Processing Systems*, vol. 33, pp. 19 088–19 098, 2020.
- [118] B. Colson, P. Marcotte, and G. Savard, "An overview of bilevel optimization," *Annals OR*, vol. 153, pp. 235–256, 06 2007.
- [119] D. Ho, E. Liang, X. Chen, I. Stoica, and P. Abbeel, "Population based augmentation: Efficient learning of augmentation policy schedules," in *International Conference on Machine Learning*. PMLR, 2019, pp. 2731–2741.
- [120] H. Liu, K. Simonyan, and Y. Yang, "Darts: Differentiable architecture search," in *ICLR*, 2019.
- [121] A. Liu, Z. Huang, Z. Huang, and N. Wang, "Direct differentiable augmentation search," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 12 219–12 228.
- [122] S. Mounsaveng, I. Laradji, I. Ben Ayed, D. Vazquez, and M. Pedersoli, "Learning data augmentation with online bilevel optimization for image classification," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021, pp. 1691–1700.
- [123] S. Lim, I. Kim, T. Kim, C. Kim, and S. Kim, "Fast autoaugment," *Advances in Neural Information Processing Systems*, vol. 32, pp. 6665–6675, 2019.
- [124] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, ser. ICML'17. JMLR.org, 2017, p. 214–223.
- [125] Y. Zheng, Z. Zhang, S. Yan, and M. Zhang, "Deep autoaugmentation," in *ICLR*, 2022.
- [126] X. Xu, H. Zhao, and P. Torr, "Universal adaptive data augmentation," *arXiv preprint arXiv:2207.06658*, 2022.
- [127] A. Fawzi, H. Samulowitz, D. Turaga, and P. Frossard, "Adaptive data augmentation for image classification," in *2016 IEEE International Conference on Image Processing (ICIP)*, 2016, pp. 3688–3692.
- [128] N. Miao, T. Rainforth, E. Mathieu, Y. Dubois, Y. W. Teh, A. Foster, and H. Kim, "Instance-specific augmentation: Capturing local invariances," in *Proceedings of the 40th International Conference on Machine Learning*, 2023.
- [129] V. Chinbat and S.-H. Bae, "Ga3n: Generative adversarial autoaugment network," *Pattern Recognition*, vol. 127, p. 108637, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320322001182>
- [130] T.-H. Cheung and D.-Y. Yeung, "AdaAug: Learning class-and instance-adaptive data augmentation policies," in *International Conference on Learning Representations*, 2021.
- [131] A. Xiao, B. Shen, J. Tian, and Z. Hu, "Differentiable randaugment: Learning selecting weights and magnitude distributions of image transformations," *IEEE Transactions on Image Processing*, 2023.
- [132] K. Bi, C. Hu, L. Xie, X. Chen, L. Wei, and Q. Tian, "Stabilizing darts with amended gradient estimation on architectural parameters," *arXiv preprint arXiv:1910.11831*, 2019.
- [133] R. Balestrieri, L. Bottou, and Y. LeCun, "The effects of regularization and data augmentation are class dependent," *arXiv preprint arXiv:2204.03632*, 2022.
- [134] J. Cohen, E. Rosenfeld, and Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *International Conference on Machine Learning*. PMLR, 2019, pp. 1310–1320.
- [135] B. Li, C. Chen, W. Wang, and L. Carin, "Certified adversarial robustness with additive noise," *Advances in neural information processing systems*, vol. 32, 2019.
- [136] C. Xie, J. Wang, Z. Zhang, Z. Ren, and A. Yuille, "Mitigating adversarial effects through randomization," *arXiv preprint arXiv:1711.01991*, 2017.
- [137] A. Sablayrolles, M. Douze, C. Schmid, Y. Ollivier, and H. Jégou, "White-box vs black-box: Bayes optimal strategies for membership inference," in *International Conference on Machine Learning*. PMLR, 2019, pp. 5558–5567.
- [138] T. Geffner and J. Domke, "A rule for gradient estimator selection, with an application to variational inference," in *International Conference on Artificial Intelligence and Statistics*. PMLR, 2020, pp. 1803–1812.
- [139] S. Ravuri and O. Vinyals, "Seeing is not necessarily believing: Limitations of biggans for data augmentation," in *ICLR*, 2019.
- [140] S. Wu, H. Zhang, G. Valiant, and C. Ré, "On the generalization effects of linear transformations in data augmentation," in *International Conference on Machine Learning*. PMLR, 2020, pp. 10 410–10 420.



deep learning, neural networks, computer vision, and image processing.

Jian Shi received his master's degree from the University of Leicester, UK. He is currently working toward the Ph.D. degree with King Abdullah University of Science and Technology, Thuwal, Saudi Arabia. Before that, he was an associate researcher at NEC laboratories, Beijing, China, working on medical computer vision technologies. Meanwhile, he is also the primary contributor to Kornia, a widely used open-source repository for computer vision and image processing for deep learning. His research interests include



Hakim Ghazzai (SM'20) received the Ph.D. degree in electrical engineering from King Abdullah University of Science and Technology (KAUST), Saudi Arabia, in 2015, and the Diplôme d'Ingenieur (Hons.) in telecommunication engineering and the master's degree in high-rate transmission systems from the Ecole Supérieure des Communications de Tunis (SUP'COM), Tunis, Tunisia, in 2010 and 2011, respectively. He was a Researcher Scholar with the Qatar Mobility Innovations Center (QMIC), Qatar, Karlstad University, Sweden, and Stevens Institute of Technology, NJ, USA. He is currently a Research Scientist with KAUST. Since 2019, he has been on the Editorial Board of the IEEE Communications Letters and the IEEE Open Journal of the Communications Society. Since 2020, he joined the Board of IoT and Sensor Networks (specialty section of Frontiers in Communications and Networks) as an associate editor. He is a recipient of appreciation for being an exemplary reviewer for IEEE Wireless Communications Letters in 2016 and IEEE Communications Letters in 2017. He is the author and co-author of more than 170 publications. His general research interests include artificial intelligence-enabled applications, the Internet of Things, Intelligent Transportation Systems (ITS), and mobile and wireless networks.



Yehia Massoud (F'15) received the Ph.D. degree in electrical engineering and computer science from the Massachusetts Institute of Technology (MIT), Cambridge, MA, USA. He is currently the Director of the Innovative Technologies Laboratories (ITL), King Abdullah University of Science and Technology, Thuwal, Saudi Arabia. He has held several positions at leading institutions of higher education and the industry including Rice University, Houston, TX, USA, Stevens Institute of Technology, Hoboken, NJ, USA, WPI, UAB, the SLAC National Accelerator Laboratory, and Synopsys Inc. From January 2018 to July 2021, he was the Dean of the School of Systems and Enterprises (SSE), Stevens Institute of Technology. Prior to Stevens, he was the Head of the Department of Electrical and Computer Engineering (ECE), Worcester Polytechnic Institute, Worcester, MA, between 2012 and 2017. In 2003, he joined Rice University as an Assistant Professor, where he became one of the fastest Rice Faculty to be granted tenure in electrical and computer engineering and computer science in 2007. His research interests include the design of state-of-the-art innovative technological solutions that span over a broad range of technical areas including smart cities, autonomy, smart health, smart mobility, embedded systems, nanophotonics, and spintronics.